

Flight Software Simulation

Software-Bus Architecture for a Simplified Avionics System

Design Document

Pablo Buceta Santos

September 21, 2026

Contents

1	Introduction & Motivation	2
2	Overall Architecture	2
2.1	High-level data paths	2
3	Frozen Module List and Responsibilities	2
3.1	Software Bus	2
3.2	Scheduler	3
3.3	Command Manager	3
3.4	Telemetry Manager	3
3.5	GNC (Guidance, Navigation & Control)	4
3.6	Health Manager	4
3.7	Memory Manager	4
3.8	Instrument Manager	5
3.9	Drivers Layer	5
3.10	Instruments	5
3.11	Communication Interface	5
4	Message Catalogue	6
4.1	Design rationale	6
4.2	Common message header	6
4.3	Command field	6
4.4	What is intentionally omitted	7
4.5	Future extension points	7
5	Scheduling & Execution Model	7
6	Build & Run Instructions	7
7	Demonstration Scenarios	7
8	Limitations & Future Work	7
9	Conclusion	8

1 Introduction & Motivation

This document describes the design of a **software-bus-based flight software simulation** developed as a personal project in preparation for a Flight Software Engineer interview at Rocket Lab (Electron).

The simulation implements a classic spacecraft avionics architecture centred on a Software Bus. Its purpose is to explore and demonstrate the interactions between typical flight-software components (command handling, telemetry, GNC, health monitoring, scheduling, etc.) in a clean, well-structured C codebase.

Important scope statement: This is an *architectural and functional simulation*, not a flight-ready or hard-real-time system. It prioritises clarity of component boundaries, message flows and design decisions over real-time guarantees, redundancy or certification artefacts.

2 Overall Architecture

The system follows a classic **software-bus** (publish/subscribe) architecture commonly found in spacecraft flight software.

[Architecture diagram – to be inserted]

All major components communicate exclusively through a central **Software Bus**. This decouples producers from consumers and makes the data flow explicit and inspectable.

2.1 High-level data paths

- **Command path:** External world → Communication Interface → Command Manager → Software Bus → target component(s)
- **Telemetry path:** Components / Instruments → Telemetry Manager → Software Bus → Communication Interface → External world
- **Health path:** Components publish heartbeats / status → Health Manager → Software Bus (alerts / safe-mode flags)
- **Control path:** GNC subscribes to sensor data and commands, publishes control outputs and vehicle state

3 Frozen Module List and Responsibilities

The following module list is considered **frozen** for the first complete version of the simulation. Responsibilities may be refined in wording, but the set of modules and their primary duties will not change.

3.1 Software Bus

Primary role

Central communication backbone of the system.

Responsibilities

- Provide a publish/subscribe (or typed message) interface.
- Route messages from publishers to all current subscribers of a given topic / message type.
- Guarantee in-order delivery within a single topic for the simulation (best-effort is acceptable).

- Allow components to register and deregister subscriptions at start-up (dynamic subscription is optional).
- Provide a simple, deterministic API usable from pure C.

Non-responsibilities

Message persistence across restarts, network distribution, prioritisation beyond simple ordering.

3.2 Scheduler

Primary role

Drive the temporal behaviour of the simulation.

Responsibilities

- Maintain a simple cyclic executive or priority-based dispatch loop.
- Call each component's periodic "tick" function at a defined rate (or deliver pending messages).
- Support both periodic tasks and aperiodic / event-driven work.
- Provide a global simulation time source.
- Allow the system to be stepped (useful for deterministic demos and testing).

Non-responsibilities

Hard real-time guarantees, multi-core scheduling, deadline miss recovery.

3.3 Command Manager

Primary role

Entry point and validator for all telecommands.

Responsibilities

- Receive raw commands from the Communication Interface.
- Perform basic validation (opcode range, parameter length, simple sanity checks).
- Translate validated commands into internal bus messages.
- Publish command messages onto the Software Bus for the intended destination component(s).
- Optionally keep a short command history / log for debugging.

Non-responsibilities

Complex authentication, encryption, or long-term command storage.

3.4 Telemetry Manager

Primary role

Collection, packaging and egress of telemetry.

Responsibilities

- Subscribe to telemetry-related topics on the Software Bus.
- Aggregate or sample data from multiple producers.
- Package telemetry into well-defined packets (or frames).

- Forward packaged telemetry to the Communication Interface.
- Support basic filtering or rate limiting if needed for demos.

Non-responsibilities

On-board storage of large telemetry archives, complex CCSDS framing (can be added later).

3.5 GNC (Guidance, Navigation & Control)

Primary role

Maintain a simplified vehicle state and produce control outputs.

Responsibilities

- Subscribe to relevant sensor data and telecommands.
- Maintain a minimal internal state vector (position, velocity, attitude, or a reduced subset).
- Run a simple control law or guidance algorithm (even a trivial proportional controller is acceptable for v1).
- Publish vehicle state and control commands onto the bus.
- Expose key state variables as telemetry.

Non-responsibilities

High-fidelity orbital dynamics, Kalman filtering, or production-grade GNC algorithms.

3.6 Health Manager

Primary role

System health monitoring and basic fault awareness.

Responsibilities

- Subscribe to heartbeat / status messages from other components.
- Detect missing heartbeats or explicit error flags.
- Maintain an overall system health state (e.g. nominal / degraded / safe).
- Publish health status and alerts onto the Software Bus.
- Optionally trigger a simple safe-mode reaction (e.g. inhibit actuators).

Non-responsibilities

Full FDIR (Fault Detection, Isolation and Recovery), redundant voting, or permanent failure logging.

3.7 Memory Manager

Primary role

Controlled memory allocation and usage tracking for the simulation.

Responsibilities

- Provide a simple deterministic allocator or memory pools for components that need dynamic memory.
- Track total and per-component memory usage.
- Publish memory statistics as telemetry.

- Detect and report allocation failures to the Health Manager.

Non-responsibilities

Full heap implementation with fragmentation handling of production quality, virtual memory, or MMU simulation.

3.8 Instrument Manager

Primary role

High-level abstraction and coordination of instruments.

Responsibilities

- Translate bus messages into driver-level requests.
- Aggregate data from multiple instruments when needed.
- Enforce simple instrument mode management (on/off, standby, etc.).
- Publish instrument data and status onto the Software Bus.

Non-responsibilities

Direct hardware register access (this belongs to the Drivers Layer).

3.9 Drivers Layer

Primary role

Low-level simulated device drivers.

Responsibilities

- Implement a thin interface that looks like a hardware driver (init, read, write, ioctl-style).
- Interact with the simulated Instruments.
- Perform basic error injection if requested (for health-manager demos).
- Remain as simple and deterministic as possible.

Non-responsibilities

Real device drivers, interrupt simulation, or DMA.

3.10 Instruments

Primary role

Simulated sensors and actuators.

Responsibilities

- Produce plausible sensor data (IMU, simple GPS-like position, temperature, etc.) or accept actuator commands.
- Support basic failure modes (stuck value, noise, complete failure) for demonstration purposes.
- Be driven exclusively through the Drivers Layer.

Non-responsibilities

High-fidelity physical models or visual simulation.

3.11 Communication Interface

Primary role

Boundary between the flight software and the external world (“ground”).

Responsibilities

- Accept telecommands from an external source (TCP socket, named pipe, or stdin for early demos).

- Deliver telemetry packets to the external world.
- Perform minimal framing / de-framing if needed.
- Remain replaceable so that different transport mechanisms can be swapped later.

Non-responsibilities

Full CCSDS TM/TC stacks, encryption, or authentication.

4 Message Catalogue

This section defines the internal message format used on the Software Bus. The format is deliberately simpler than a full CCSDS Space Packet while still retaining the concepts that are educationally valuable for a flight-software simulation.

4.1 Design rationale

Internal bus messages only travel between components inside the simulation. They do not cross a radio link. Consequently they prioritise clarity, ease of construction, and low overhead over full protocol compliance.

A richer, CCSDS-inspired layout may later be introduced at the Communication Interface when messages leave the system toward the “ground”. That framing is considered out of scope for the first version of the internal bus.

4.2 Common message header

Every message carried by the Software Bus contains the following fields:

Field	Purpose	Notes
Type	Distinguishes the nature of the message	Examples: Command, Telemetry,
Message ID	Routing key used by the Software Bus	The primary value the bus match scriptions
App ID	Identifier of the publishing component	Useful for debugging, health monit ing
Timestamp & Sequence number	Per-source (or per-topic) counter	Helps detect gaps or reordering d
Length	Size of the payload in bytes	Required for variable-length paylo
Command field	Action specifier for command messages	Present when Type = Command requested action
Payload	Message-specific data	Interpretation depends on Topic

4.3 Command field

The Command field is kept as a distinct member of the common header rather than being buried inside the payload. Its purpose is to name the action the receiving component should perform (for example “set mode”, “fire thruster”, “reset counters”, etc.).

This separation makes command handling clearer: a component can inspect the Command field first and then interpret the payload according to that action. Telemetry and health messages leave this field unused or set to a neutral value.

4.4 What is intentionally omitted

The following CCSDS-style elements are *not* part of the internal message format:

- Version number and secondary-header flags
- Full Application Process ID (APID) bit-packing
- Long telemetry secondary headers
- Checksums or authentication fields

These may appear later in an external packet format produced by the Communication Interface, but they are not required for component-to-component communication inside the simulation.

4.5 Future extension points

The header is expected to remain stable for the first complete version of the simulation. Possible later additions (explicitly optional) include a timestamp field or a simple flags word. Any such addition will be documented here before it is introduced in code.

5 Scheduling & Execution Model

(To be filled once the Scheduler design is finalised.)

6 Build & Run Instructions

(To be completed when the repository skeleton exists.)

7 Demonstration Scenarios

Planned scenarios (subject to refinement):

1. Nominal command → actuation → telemetry path.
2. Simple GNC closed-loop behaviour.
3. Heartbeat loss detected by Health Manager.
4. Memory allocation pressure / failure reporting.
5. Instrument failure injection and resulting health alert.

8 Limitations & Future Work

- Single-process, non-real-time simulation on Linux.
- Simplified GNC and instrument models.
- No redundancy, voting, or formal FDIR.
- No network distribution of the Software Bus.

Possible future extensions (explicitly out of current scope): multi-process version, basic RTOS integration, more realistic dynamics, CCSDS-inspired framing, etc.

9 Conclusion

(To be written at the end of the project.)